Mini-SIEM : détecter des attaques web dans des journaux nginx

Rapport de projet personnel · Ayoub BOUSSOUKAYA · novembre 2026 · laboratoire local et éducatif (aucune cible réelle)

1. Objectif

Transformer des journaux de connexion en données exploitables pour repérer des attaques web simples, en reliant **données** (ETL, entrepôt en étoile), **cybersécurité** (comprendre les traces d'une attaque) et **développement** (API, tests, Docker).

2. Méthode

- **Laboratoire Docker** : OWASP Juice Shop (application volontairement vulnérable) derrière un reverse proxy nginx qui journalise chaque requête ; PostgreSQL ; ports liés à `127.0.0.1` uniquement.
- **Trafic de test** (`attack_sim.py`) : 15 visiteurs normaux et 5 scénarios d'attaque, chacun avec une IP fictive envoyée dans `X-Forwarded-For`. Un garde-fou refuse toute cible autre que localhost (testé, y compris `localhost.evil.com` et `localhost@evil.com`).
- **ETL** (`etl.py`) : expression régulière sur le format *combined*, décodage des URL (`%27` → `'`), rejet des lignes illisibles, chargement de `staging` puis du schéma en étoile (`fait_requete` + dimensions temps, IP, URL, statut) en une seule transaction. Relancer donne le même résultat (idempotent).
- **Détection** (`detect.py`) : 2 règles par comptage par IP et par minute (R1, R4) et 3 règles par motif sur l'URL décodée deux fois (R2, R3, R5). **30 tests pytest**, dont les seuils testés à la limite (10 échecs → rien, 11 → alerte).
- **Restitution** : API FastAPI en lecture seule (requêtes SQL paramétrées) et dashboard Streamlit qui passe par l'API (sans le mot de passe de la base).

3. Résultats (882 requêtes, dont 575 de visiteurs normaux)

Règle	Seuil / motif	Alertes	Verdict
R1 brute force (A07)	> 10 × 401 sur le login / IP / min	2	les 2 scénarios ; erreurs isolées ignorées
R2 injection SQL (A03)	`UNION SELECT`, `OR 1=1`, `--`…	8 hautes	toutes les injections détectées
	apostrophe seule	36 faibles	**faux positifs** : 36 recherches « l'orange »
R3 XSS (A03)	`<script`, `javascript:`, `onerror=`	12	toutes détectées
R4 scan	> 30 × 404 / IP / min	2	scanner : 80 × 404/min ; visiteur : 3 au maximum
R5 traversée (A01)	`../` après double décodage	23	toutes détectées, mais 13 attribuées à l'IP de Docker

> **Bilan :** chaque IP attaquante est détectée par la bonne règle et **aucune alerte haute ne vise un visiteur normal**. Les seuls faux positifs sont classés en gravité faible : c'est un choix assumé, car une apostrophe seule est souvent légitime, mais l'ignorer laisserait passer des injections simples.

4. Limites

- **Seule l'adresse est visible** : nginx n'écrit pas le corps des POST (mot de passe, injection dans un formulaire).
- **IP non fiables** : `X-Forwarded-For` est accepté de tous (labo uniquement). Une traversée qui sort de la racine est rejetée par nginx (400) avant la lecture des en-têtes, donc elle est journalisée avec l'IP de Docker.
- **Fenêtre fixe d'une minute** : 8 échecs à hh:mm:55 puis 8 à hh:mm+1:05 passent inaperçus.
- **R4 ne compte que les 404** : Juice Shop renvoie 200 pour `/admin.php`, donc 20 % du scan échappe au comptage.
- Traitement par lots ; une alerte par requête ; triple encodage non décodé.

5. Ce qui reste à faire

- Fenêtre glissante de 60 s pour R1 et R4 (`rolling("60s")` en pandas).
- Compter aussi les 403 et les pages 200 inhabituelles pour R4.
- Regrouper les alertes par IP et par période pour réduire le bruit.
- Journaliser le corps des POST de connexion (F12).
- Comparer avec un scan OWASP ZAP sur le labo (F11).
- Chargement incrémental (nouvelles lignes seulement).

6. Ce que j'ai appris

Les journaux ne contiennent pas tout : il faut savoir ce qu'ils montrent (statut, URL) et ce qu'ils cachent (corps, vraie IP). Deux bugs instructifs : un journal corrompu par des octets nuls (fichier vidé pendant que nginx écrivait) et une règle qui plantait sur les vraies données (`NaN` au lieu de `None` en pandas), corrigée en écrivant d'abord le test qui reproduit le bug.